```
#include <iostream>
#include <cstdlib>
Using namespace std;

Struct Node
{
   Int id;
   Char name[50];
   Char status[20];
   Node *next;
};

Node *head = NULL;
Node *top = NULL;
Node *front = NULL;
Node *rear = NULL;

// Input Item Details
Void inputItem(Node *n)
{
   Cout << “Enter Item ID: “;
   Cin >> n->id;

   Cout << “Enter Item Name: “;
   Cin >> ws;
   Cin.getline(n->name, 50);
```

```
    Cout << “Enter Status (Lost/Found): “;
    Cin >> n->status;
}

// 1. Add Item to Linked List
Void addItem()
{
    Node *n = new Node;
    inputItem(n);
    n->next = NULL;

    if (head == NULL)
        head = n;
    else
    {
        Node *temp = head;
        While (temp->next != NULL)
            Temp = temp->next;

        Temp->next = n;
    }

    Cout << “Item Added Successfully!\n”;
}
```

```
// 2. Search Item
Void searchItem()
{
   Int id;
   Cout << “Enter Item ID: “;
   Cin >> id;

   Node *temp = head;

   While (temp != NULL)
   {
      If (temp->id == id)
      {
         Cout << “Name: “ << temp->name
            << “\nStatus: “ << temp->status << endl;
         Return;
      }
      Temp = temp->next;
   }

   Cout << “Item Not Found!\n”;
}

// 3. Display Items
Void displayItems()
{
```

```
    Node *temp = head;

    If (temp == NULL)
    {
        Cout << “No Items Available!\n”;
        Return;
    }

    While (temp != NULL)
    {
        Cout << “\nID: “ << temp->id
            << “\nName: “ << temp->name
            << “\nStatus: “ << temp->status << endl;

        Temp = temp->next;
    }
}

// 4. Remove Item
Void removeItem()
{
    Int id;
    Cout << “Enter Item ID to Remove: “;
    Cin >> id;

    Node *temp = head;
```

```
    Node *prev = NULL;

    While (temp != NULL && temp->id != id)
    {
        Prev = temp;
        Temp = temp->next;
    }

    If (temp == NULL)
    {
        Cout << “Item Not Found!\n”;
        Return;
    }

    If (prev == NULL)
        Head = temp->next;
    Else
        Prev->next = temp->next;

    Delete temp;
    Cout << “Item Removed Successfully!\n”;
}

// 5. Stack Push
Void push()
{
```

```
    Node *n = new Node;
    inputItem(n);

    n->next = top;
    top = n;

    cout << “Item Pushed into Stack!\n”;
}

// 6. Stack Pop
Void pop()
{
    If (top == NULL)
    {
        Cout << “Stack Underflow!\n”;
        Return;
    }

    Node *temp = top;
    Cout << “Removed: “ << temp->name << endl;

    Top = top->next;
    Delete temp;
}

// 7. Display Stack
```

```
Void displayStack()
{
   Node *temp = top;

   If (temp == NULL)
   {
      Cout << “Stack is Empty!\n”;
      Return;
   }

   While (temp != NULL)
   {
      Cout << temp->id << “ – “ << temp->name << endl;
      Temp = temp->next;
   }
}

// 8. Queue Enqueue
Void enqueue()
{
   Node *n = new Node;
   inputItem(n);
   n->next = NULL;

   if (rear == NULL)
      front = rear = n;
```

```
    else
    {
        Rear->next = n;
        Rear = n;
    }

    Cout << “Item Added to Queue!\n”;
}

// 9. Queue Dequeue
Void dequeue()
{
    If (front == NULL)
    {
        Cout << “Queue Underflow!\n”;
        Return;
    }

    Node *temp = front;
    Cout << “Processed: “ << temp->name << endl;

    Front = front->next;

    If (front == NULL)
        Rear = NULL;
```

```
   Delete temp;
}

// 10. Display Queue
Void displayQueue()
{
   Node *temp = front;

   If (temp == NULL)
   {
      Cout << “Queue is Empty!\n”;
      Return;
   }

   While (temp != NULL)
   {
      Cout << temp->id << “ – “ << temp->name << endl;
      Temp = temp->next;
   }
}

// 11. Calloc() Demonstration
Void callocDemo()
{
   Int *arr = (int *)calloc(5, sizeof(int));
```

```
   If (arr == NULL)
   {
      Cout << “Memory Allocation Failed!\n”;
      Return;
   }

   Cout << “Array values: “;

   For (int I = 0; I < 5; i++)
      Cout << arr[i] << “ “;

   Cout << “\nMemory allocated using calloc().\n”;

   Free(arr);
}

// Main Function
Int main()
{
   Int choice;

   Do
   {
      Cout << “\n--- College Lost & Found System ---\n”;
      Cout << “1. Add Item\n”;
      Cout << “2. Search Item\n”;
```

```
Cout << “3. Display Items\n”;
Cout << “4. Remove Item\n”;
Cout << “5. Stack Push\n”;
Cout << “6. Stack Pop\n”;
Cout << “7. Display Stack\n”;
Cout << “8. Queue Enqueue\n”;
Cout << “9. Queue Dequeue\n”;
Cout << “10. Display Queue\n”;
Cout << “11. Calloc Demonstration\n”;
Cout << “0. Exit\n”;
Cout << “Enter Choice: “;
Cin >> choice;

Switch (choice)
{
   Case 1: addItem(); break;
   Case 2: searchItem(); break;
   Case 3: displayItems(); break;
   Case 4: removeItem(); break;
   Case 5: push(); break;
   Case 6: pop(); break;
   Case 7: displayStack(); break;
   Case 8: enqueue(); break;
   Case 9: dequeue(); break;
   Case 10: displayQueue(); break;
   Case 11: callocDemo(); break;
```

```
        Case 0: cout << “Thank You!\n”; break;
        Default: cout << “Invalid Choice!\n”;
      }

   } while (choice != 0);

   Return 0;
}
```